



CPE3243 วิศวกรรมซอฟต์แวร์ (Software Engineering)

Piyavit Laung-Aram
Major of Computer Engineering
Faculty of Engineering
Ramkhamhaeng University, Thailand

การเขียนโปรแกรมด้วยภาษา Java ส่วนที่ 3

(Java Programming Part 3)



Java Syntax และ Java Output

โปรแกรมตัวอย่างที่ 1- ไฟล์ Main.java

```
//Example 1
```

```
public class Main  
{
```

```
    public static void main(String[] args)  
    {  
        System.out.println("Hello World");  
    }  
}
```

ชื่อไฟล์ .java และชื่อ class ต้องเหมือนกัน ทั้ง
ตัวพิมพ์เล็กใหญ่

ผลการรันโปรแกรม

Hello World

คุณสามารถใช้เมธอด println() เพื่อส่งออกค่า
หรือพิมพ์ข้อความใน Java



Java Syntax และ Java Output(ต่อ)

โปรแกรมตัวอย่างที่ 2- ไฟล์ Main.java

```
//Example 2
public class Main
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
        System.out.println("I am learning Java.");
        System.out.println("It is Good Language");
    }
}
```

ผลการรันโปรแกรม

```
Hello World!
I am learning Java.
It is Good Language
```

วิธีการแสดง Output บน

หน้าจอ

เมธิด print() ซึ่งคล้ายกับ

println() ข้อแตกต่างเพียง

อย่างเดียวคือไม่ขึ้นบรรทัดใหม่

คุณสามารถใช้เมธิด println() เพื่อส่งออกค่า
หรือพิมพ์ข้อความใน Java



Java Comments

โปรแกรมตัวอย่างที่ 3- ไฟล์ Main.java

```
//Example 3
public class Main
{
    public static void main(String[] args)
    {
        // EXample 3 java comments

        /* The code below will print the words Hello World
        to the screen, and it is amazing */
        System.out.println("Hello World");
    }
}
```

ผลการรันโปรแกรม

Hello World

ความคิดเห็นสามารถใช้เพื่ออธิบายโค้ด
ภาษา Java จะช่วยให้การอ่านโค้ดง่าย
ขึ้น นอกจากนี้ยังสามารถใช้เพื่อป้องกัน
ข้อผิดพลาดในการทดสอบโค้ดได้

ความคิดเห็นบรรทัดเดียวเริ่มต้นด้วย
เครื่องหมายทับสองอัน (//) ข้อความใดๆ
ระหว่าง // และท้ายบรรทัดจะถูกละเว้น
โดย Java (จะไม่ถูกดำเนินการ)

ความคิดเห็นหลายบรรทัดขึ้นต้นด้วย /*
และลงท้ายด้วย */ ข้อความใดๆ ระหว่าง
/* และ */ จะถูกละเว้นโดย Java



Java Variables

ตัวแปรคือคอนเทนเนอร์สำหรับเก็บค่าข้อมูลใน Java มีตัวแปรประเภทต่างๆ เช่น

String - เก็บข้อความ เช่น "Hello" ค่าสตริงถูกล้อมรอบด้วยเครื่องหมายคำพูดคู่

int - เก็บจำนวนเต็ม (จำนวนเต็ม) โดยไม่มีทศนิยม เช่น 123 หรือ -123

float - เก็บเลขทศนิยมด้วยทศนิยม เช่น 19.99 หรือ -19.99

double- เก็บเลขทศนิยมด้วยทศนิยม 2 เท่าของ float

char - เก็บอักขระตัวเดียว เช่น 'a' หรือ 'B' ค่า char ถูกล้อมรอบด้วยเครื่องหมาย single quotes

boolean - เก็บค่าด้วยสองสถานะ: จริงหรือเท็จ

Syntax

```
type variableName = value;
```



Java Variables (ต่อ)

โปรแกรมตัวอย่างที่ 4- ไฟล์ Main.java

```
//Example 4
public class Main
{
    public static void main(String[] args)
    {
        String var1 = "Hello";
        int var2 = 10;
        float var3 = 10.22f;
        double var4 = 10.22d;
        boolean var5 = true;
        char var6 = 'a';

        System.out.println(var1);
        System.out.println(var2);
        System.out.println(var3);
        System.out.println(var4);
        System.out.println(var5);
        System.out.println(var6);
    }
}
```

ผลการรันโปรแกรม

```
Hello
10
10.22
10.22
true
a
```



Java Variables (ต่อ)-Final Variables

ถ้าคุณไม่ต้องการให้ผู้อื่น (หรือตัวคุณเอง) เขียนทับค่าที่มีอยู่ ให้ใช้คีย์เวิร์ด `final` (ซึ่งจะประกาศตัวแปรเป็น “final” หรือ “constant” ซึ่งหมายถึงไม่สามารถเปลี่ยนแปลงได้และอ่านอย่างเดียว):

โปรแกรมตัวอย่างที่ 5- ไฟล์ Main.java

```
//Example 5
public class Main
{
    public static void main(String[] args)
    {
        final int myNumber = 10;
        myNumber = 50; // This code will generate an error.
        System.out.println(myNumber);
    }
}
```

ผลการรันโปรแกรม

```
Main.java:6: error: cannot assign a value to final variable myNumber
    myNumber = 50; // This code will generate an error.
    ^
```

1 error



Java Variables (ต่อ)-Declare Many Variables

หากต้องการประกาศตัวแปรประเภทเดียวกันมากกว่าหนึ่งตัว คุณสามารถใช้รายการคั่นด้วยเครื่องหมายจุลภาค

โปรแกรมตัวอย่างที่ 6- ไฟล์ Main.java

```
//Example 6
public class Main
{
    public static void main(String[] args)
    {
        int x = 10, y = 10, z = 30;
        System.out.println(x + y + z);
    }
}
```

ผลการรันโปรแกรม



Java Variables (ต่อ)-One Value to Multiple Variables

โปรแกรมตัวอย่างที่ 7- ไฟล์ Main.java

```
//Example 7
public class Main
{
    public static void main(String[] args)
    {
        int x, y, z;
        x = y = z = 100;
        System.out.println(x + y + z);
    }
}
```

ผลการรันโปรแกรม

300



Java Variables (ต่อ)-Identifiers

ตัวแปร Java ทั้งหมดต้องระบุด้วยชื่อที่ไม่ซ้ำ ชื่อเฉพาะเหล่านี้เรียกว่าตัวระบุ-Identifiers ตัวระบุอาจเป็นชื่อย่อ (เช่น x และ y) หรือชื่อที่สื่อความหมาย (ผลรวม TotalValue)

หมายเหตุ: ขอแนะนำให้อ่านชื่อที่สื่อความหมายเพื่อสร้างโค้ดที่เข้าใจและดูแลรักษาได้

กฎทั่วไปสำหรับการตั้งชื่อตัวแปรคือ

ชื่อสามารถประกอบด้วยตัวอักษร ตัวเลข ขีดล่าง และเครื่องหมายดอลลาร์

ชื่อต้องขึ้นต้นด้วยตัวอักษร-แ่นะใช้ตัวเล็ก ถ้ามีหลายคำคำถัดไปตัวอักษรแรกเป็นตัวใหญ่

เช่น myValue เนื่องจากชื่อคลาสจะใช้ตัวใหญ่

ชื่อควรขึ้นต้นด้วยตัวพิมพ์เล็กและต้องไม่มีช่องว่าง

ชื่อสามารถเริ่มต้นด้วย \$ และ _ (แต่เราจะไม่ใช้ในบทช่วยสอนนี้)

ชื่อจะคำนึงถึงขนาดตัวพิมพ์ ("myValue" และ "myvalue" เป็นตัวแปรที่แตกต่างกัน)

คำสงวน (เช่นคำหลัก Java เช่น int หรือ boolean) ไม่สามารถใช้เป็นชื่อได้



Java Data Types

ตามที่อธิบายไว้ในเนื้อหา Java Variables ตัวแปรในภาษา Java ต้องระบุชนิดข้อมูล-data types

```
String var1 = "Hello";  
int var2 = 10;  
float var3 = 10.22f;  
double var4 = 10.22;  
boolean var5 = true;  
char var6 = 'A';
```

ชนิดข้อมูล-data types แบ่งออกเป็นสองกลุ่ม

- ชนิดข้อมูลดั้งเดิม(Primitive data types) - ประกอบด้วย byte, short, int, long, float, double, boolean และ char
- ชนิดข้อมูลที่ไม่ใช่แบบพื้นฐาน(Non-primitive data types) - เช่น String, Arrays และ Classes (คุณจะได้เรียนรู้เพิ่มเติมเกี่ยวกับสิ่งเหล่านี้ในลำดับถัดไป)



Java Data Types (ต่อ)

ชนิดข้อมูล (Data Type)	ขนาด(Size)	คำอธิบาย(Description)
byte	1 byte	เก็บตัวเลขทั้งหมดจาก -128 ถึง 127
short	2 bytes	เก็บตัวเลขทั้งหมดจาก -32,768 ถึง 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	เก็บตัวเลขทั้งหมดจาก -9,223,372,036,854,775,808 ถึง 9,223,372,036,854,775,807
float	4 bytes	เก็บตัวเลขเศษส่วน เพียงพอสำหรับเก็บทศนิยม 6 ถึง 7 หลัก
double	8 bytes	เก็บตัวเลขเศษส่วน เพียงพอสำหรับเก็บทศนิยม 15 หลัก
boolean	1 bit	เก็บค่าจริงหรือเท็จ
char	2 bytes	เก็บอักขระ/ตัวอักษรเดียวหรือค่า ASCII



Java Numbers

ตัวเลข-Numbers

ประเภทตัวเลขดั้งเดิมแบ่งออกเป็นสองกลุ่ม คือ

- ประเภทจำนวนเต็มจะเก็บเก็บจำนวนเต็ม บวกหรือลบ (เช่น 123 หรือ -456) โดยไม่มีทศนิยม ประเภทที่ถูกต้องคือ ไบต์ สั้น int และยาว คุณควรใช้ประเภทใดขึ้นอยู่กับค่าตัวเลข
- ประเภทจุดทศนิยมแสดงถึงตัวเลขที่มีเศษส่วนซึ่งมีทศนิยมตั้งแต่หนึ่งตำแหน่งขึ้นไป มีสองประเภท: ลอยและคู่

แม้ว่า Java จะมีตัวเลขหลายประเภท แต่ตัวเลขที่ใช้มากที่สุดคือ int (สำหรับจำนวนเต็ม) และ double (สำหรับตัวเลขทศนิยม)



Java Numbers(ต่อ)-Integer Types

byte

ชนิดข้อมูลไบต์สามารถเก็บจำนวนเต็มได้ตั้งแต่ -128 ถึง 127 สามารถใช้แทน int หรือ ชนิดจำนวนเต็มอื่น ๆ เพื่อบันทึกหน่วยความจำ เมื่อคุณแน่ใจว่าค่าจะอยู่ภายใน -128 และ 127

โปรแกรมตัวอย่างที่ 8- ไฟล์ Main.java

```
//Example 8
public class Main
{
    public static void main(String[] args)
    {
        byte myNumber = 100;
        System.out.println(myNumber);
    }
}
```

ถ้าแก้เป็น 200 ละ

ผลการรันโปรแกรม

100



Java Numbers(ต่อ)-Integer Types

short

ชนิดข้อมูลแบบ **short** สามารถเก็บตัวเลขทั้งหมดได้ตั้งแต่ -32768 ถึง 32767:

โปรแกรมตัวอย่างที่ 9- ไฟล์ Main.java

```
//Example 9
public class Main
{
    public static void main(String[] args)
    {
        short myNumber = 5000;
        System.out.println(myNumber);
    }
}
```

ถ้าแก้เป็น 33000 ละ

ผลการรันโปรแกรม

5000



Java Numbers(ต่อ)-Integer Types

int

ชนิดข้อมูล int สามารถเก็บจำนวนเต็มได้ตั้งแต่ -2147483648 ถึง 2147483647 โดยทั่วไป
ชนิดข้อมูล int เป็นชนิดข้อมูลที่ต้องการเมื่อเราสร้างตัวแปรด้วยค่าตัวเลข

โปรแกรมตัวอย่างที่ 10- ไฟล์ Main.java

```
//Example 10
public class Main
{
    public static void main(String[] args)
    {
        int myNumber = 100000;
        System.out.println(myNumber);
    }
}
```

ถ้าแก้เป็น
3000000000 ละ

ผลการรันโปรแกรม

100000



Java Numbers(ต่อ)-Integer Types

long

ชนิดข้อมูลแบบ long สามารถเก็บตัวเลขทั้งหมดได้ตั้งแต่ -9223372036854775808 ถึง 9223372036854775807 ซึ่งใช้เมื่อ int มีขนาดไม่ใหญ่พอที่จะเก็บค่า โปรดทราบว่าควรลงท้ายค่าด้วย "L"

โปรแกรมตัวอย่างที่ 11- ไฟล์ Main.java

```
//Example 11
public class Main
{
    public static void main(String[] args)
    {
        long myNumber = 15000000000L;
        System.out.println(myNumber);
    }
}
```

ถ้าแก้เป็น

15000000000 ละ

ผลการรันโปรแกรม

15000000000



Java Numbers(ต่อ)-Floating Point Types

ประเภท Floating Point หรือ ทศนิยม ควรใช้เมื่อต้องการตัวเลขที่มีทศนิยม เช่น 9.99 หรือ 3.14515 ประเภทข้อมูล float และ double สามารถเก็บตัวเลขที่เป็นเศษส่วนได้ โปรดทราบว่าควรลงท้ายค่าด้วย "f" สำหรับ float และ "d" สำหรับ double

โปรแกรมตัวอย่างที่ 12- ไฟล์ Main.java

```
//Example 12
public class Main
{
    public static void main(String[] args)
    {
        float myNumber = 5.55f;
        System.out.println(myNumber);
    }
}
```

ถ้าไม่มี f ต่อท้าย
ละ

ผลการรันโปรแกรม

5.55



Java Numbers(ต่อ)-Floating Point Types

จะใช้ float หรือ double??

ความแม่นยำของ float ระบุจำนวนหลักที่ค่าสามารถมีได้หลังจุดทศนิยม ความแม่นยำของทศนิยมมีเพียงหกหรือเจ็ดหลัก ในขณะที่ตัวแปร double มีความแม่นยำประมาณ 15 หลัก ดังนั้นจึงปลอดภัยกว่าที่จะใช้ double สำหรับการคำนวณส่วนใหญ่

โปรแกรมตัวอย่างที่ 13- ไฟล์ Main.java

```
//Example 13
public class Main
{
    public static void main(String[] args)
    {
        double myNumber = 29.99d;
        System.out.println(myNumber);
    }
}
```

ถ้าไม่มี d ต่อท้าย
ละ

ผลการรันโปรแกรม

19.88



Java Numbers(ต่อ)-Scientific Numbers

ตัวเลขทางวิทยาศาสตร์

เลขทศนิยมสามารถเป็นตัวเลขทางวิทยาศาสตร์ที่มี "e" เพื่อระบุกำลัง 10:

โปรแกรมตัวอย่างที่ 14- ไฟล์ Main.java

```
//Example 14
public class Main
{
    public static void main(String[] args)
    {
        float f1 = 35e4f;
        double d1 = 12E5d;
        System.out.println(f1);
        System.out.println(d1);
    }
}
```

ถ้าปรับตัวเลข
หลัง E ละ

ผลการรันโปรแกรม

```
350000.0
1200000.0
```



Java Boolean Data Types

boolean

ชนิดข้อมูลบูลีนถูกประกาศด้วยคีย์เวิร์ดบูลีนและสามารถรับได้เฉพาะค่าจริงหรือเท็จเท่านั้น

โปรแกรมตัวอย่างที่ 15- ไฟล์ Main.java

```
//Example 15
public class Main
{
    public static void main(String[] args)
    {
        boolean isJavaFun = true;
        boolean isFishTasty = false;
        System.out.println(isJavaFun);
        System.out.println(isFishTasty);
    }
}
```

ผลการรันโปรแกรม

```
true
false
```



Java Character

Char ชนิดข้อมูลที่ใช้เพื่อเก็บอักขระตัวเดียว อักขระต้องล้อมรอบด้วยเครื่องหมายคำพูดเดียว เช่น 'A' หรือ 'c'

โปรแกรมตัวอย่างที่ 16- ไฟล์ Main.java

```
//Example 16
public class Main
{
    public static void main(String[] args)
    {
        char myGrade = 'B';
        System.out.println(myGrade);
    }
}
```

ผลการรันโปรแกรม

B



Java Character(ต่อ)

หรือหากคุณคุ้นเคยกับค่า ASCII คุณสามารถใช้ค่าเหล่านี้เพื่อแสดงอักขระบางตัวได้:

โปรแกรมตัวอย่างที่ 17- ไฟล์ Main.java

```
//Example 17
public class Main
{
    public static void main(String[] args)
    {
        char myVar1 = 65, myVar2 = 66, myVar3 = 67;
        System.out.println(myVar1);
        System.out.println(myVar2);
        System.out.println(myVar3);
    }
}
```

ผลการรันโปรแกรม

A
B
C



Java Strings

ชนิดข้อมูลสตริงใช้เพื่อจัดเก็บลำดับของอักขระ (ข้อความ) ค่าสตริงต้องล้อมรอบด้วยเครื่องหมายคำพูดคู่

โปรแกรมตัวอย่างที่ 18- ไฟล์ Main.java

```
//Example 18
public class Main
{
    public static void main(String[] args)
    {
        String greeting = "Hello World";
        System.out.println(greeting);
    }
}
```

ผลการรันโปรแกรม

Hello World

ประเภทสตริงมีการใช้งานและบูรณาการอย่างมากใน Java ซึ่งบางคนเรียกมันว่า "ประเภทที่เก่าพิเศษ" สตริงใน Java เป็นประเภทข้อมูลที่ไม่ใช่แบบพื้นฐาน เพราะมันหมายถึงวัตถุ ออบเจกต์สตริงมีเมธอดที่ใช้ในการดำเนินการบางอย่างกับสตริง



Java Non-Primitive Data Types

ชนิดข้อมูลที่ไม่ใช่พื้นฐาน – ชนิดข้อมูลที่ไม่ใช่พื้นฐานเรียกว่าประเภทอ้างอิงเนื่องจากอ้างอิงวัตถุ ความแตกต่างหลักระหว่างชนิดข้อมูลพื้นฐานและไม่ใช่พื้นฐานคือ

- ประเภทดั้งเดิมถูกกำหนดไว้ล่วงหน้า (กำหนดไว้แล้ว) ใน Java โปรแกรมเมอร์สร้างประเภทที่ไม่ใช่แบบพื้นฐานและไม่ได้กำหนดโดย Java (ยกเว้นสตริง)
- ชนิดที่ไม่ใช่แบบพื้นฐานสามารถใช้เพื่อเรียกเมธอดเพื่อดำเนินการบางอย่างได้ ในขณะที่ประเภทดั้งเดิมไม่สามารถทำได้
- ประเภทดั้งเดิมมีค่าเสมอ ในขณะที่ประเภทที่ไม่ใช่แบบพื้นฐานสามารถเป็นค่าว่างได้
- ชนิดพื้นฐานจะเริ่มต้นด้วยอักขรตัวพิมพ์เล็ก ในขณะที่ประเภทที่ไม่ใช่แบบพื้นฐานจะเริ่มต้นด้วยอักขรตัวพิมพ์ใหญ่
- ขนาดของประเภทพื้นฐานขึ้นอยู่กับประเภทข้อมูล ในขณะที่ประเภทที่ไม่ใช่แบบพื้นฐานจะมีขนาดเท่ากันทั้งหมด
- ตัวอย่างของ non-primitive types ได้แก่ Strings, Arrays, Classes, Interface เป็นต้น คุณจะได้เรียนรู้เพิ่มเติมเกี่ยวกับสิ่งเหล่านี้ในบทต่อไป



Java Non-Primitive Data Types

Java Type Casting

Java Type Casting คือเมื่อคุณกำหนดค่าของประเภทข้อมูลพื้นฐานหนึ่งให้กับอีกประเภทหนึ่ง

ใน Java การแคสต์มีสองประเภท:

Widening Casting (อัตโนมัติ) - แปลงประเภทที่เล็กกว่าเป็นขนาดที่ใหญ่กว่า

byte -> short -> char -> int -> long -> float -> double

Narrowing Casting (ด้วยตนเอง) - การแปลงประเภทที่ใหญ่กว่าเป็นประเภทที่เล็กกว่า

double -> float -> long -> int -> char -> short -> byte



Java Type Casting(ต่อ)

Widening Casting จะกระทำโดยอัตโนมัติเมื่อส่งประเภทขนาดที่เล็กกว่าไปยังประเภทที่ใหญ่กว่า:

โปรแกรมตัวอย่างที่ 19- ไฟล์ Main.java

```
//Example 19
public class Main
{
    public static void main(String[] args)
    {
        int myInt = 9;
        double myDouble = myInt; // Automatic casting: int to double

        System.out.println(myInt);
        System.out.println(myDouble);
    }
}
```

ผลการรันโปรแกรม

9
9.0



Java Type Casting(ต่อ)

Narrowing Casting จะต้องดำเนินการด้วยตนเองโดยวางประเภทในวงเล็บไว้ข้างหน้าค่า

โปรแกรมตัวอย่างที่ 20- ไฟล์ Main.java

```
//Example 20
public class Main
{
    public static void main(String[] args)
    {
        double myDouble = 9.78d;
        int myInt = (int) myDouble; // Explicit casting: double to int

        System.out.println(myDouble);
        System.out.println(myInt);
    }
}
```

ผลการรันโปรแกรม

9.78

9

ตัวดำเนินการในภาษา Java

ตัวดำเนินการ (Operator)

ภาษา Java มีตัวดำเนินการ ดังต่อไปนี้

- **ตัวดำเนินการเลขคณิต (Arithmetic Operator)**
- **ตัวดำเนินการลอจิก (Logic Operator)**
- **ตัวดำเนินการเปรียบเทียบ (Relation Operator)**

ตัวดำเนินการเลขคณิต (Arithmetic Operator)

operator	ความหมาย	ชนิดของข้อมูล	ตัวอย่าง
+	บวก	ตามชนิดของข้อมูล	5 + 2
-	ลบ	ตามชนิดของข้อมูล	5 - 2
*	คูณ	ตามชนิดของข้อมูล	5 * 2
/	หาร	ตามชนิดของข้อมูล	5 / 2
%	หารเอาผลลัพธ์เฉพาะเศษของการหาร (Modulo)	จำนวนเต็ม	5%2
++	การเพิ่มค่าขึ้นหนึ่ง	จำนวนเต็ม	X++ , ++X
--	การลดค่าลงหนึ่ง	จำนวนเต็ม	X-- , --X

การกำหนดขนาดของข้อมูลในภาษา Java

ชนิดข้อมูล X

ชนิดข้อมูล y

X โอเปอเรชัน y

int

int

int

float

float

float

int

long

long

char

int

int

int

float

float

int

double

double

float

double

double

long

double

double

ชนิดข้อมูลต่าง ๆ

long double

long double

- - -

- - -

- - -

ตัวอย่างที่ 2

ค่าตัวแปร x	ค่าตัวแปร y	การดำเนินการ	ค่าจากการกระทำ	ผลลัพธ์ที่เก็บ
2	4	$x = y + 2$	6	$x = 6$
10	3	$x = x/y$	3	$x = 3$
10.0	3	$x = x/y$	3.333333	$x = 3.333333$
9	2	$x = x \% y$	1	$x = 1$
14	-3	$x = x \% y$	2	$x = 2$
-14	3	$x = x \% y$	-2	$x = -2$

ลำดับการทำงานของตัวดำเนินการ

ตัวดำเนินการ

ลำดับที่

()

1 (สูงที่สุด)

++ --

2

* / %

3

+ -

4

+= -= *= = /= %=

5 (ต่ำสุด)



ตาราง : แสดงลำดับการทำงานของตัวดำเนินการ

ตัวอย่างที่ 3

การดำเนินการ	ผลลัพธ์	ชนิดผลลัพธ์
$13 / 5 * 3$	6	จำนวนเต็ม
$6 * 5 / 10 * 2 + 10$	16	จำนวนเต็ม
$(6 * 5) / (10 * 2) + 10$	11	จำนวนเต็ม
$(6 * 5.0) / (10 * 2 + 10)$	1.000000	จำนวนจริง
$(6 * 5.0) / (10 * (2 + 10))$	0.250000	จำนวนจริง
$3 * (4 \% (6 / 2)) + 5$	8	จำนวนเต็ม

ตัวอย่างที่ 4

ค่าตัวแปร x	การดำเนินการ	ค่าจากการกระทำ
5	$x = x+1$	x มีค่าเป็น 6
5	$x = x-1$	x มีค่าเป็น 4
5	$x = x++$	x มีค่าเป็น 6
5	$x = ++x$	x มีค่าเป็น 6
5	$x = x--$	x มีค่าเป็น 4
5	$x = --x$	x มีค่าเป็น 4
5	$y = ++x$	y มีค่าเป็น 6
5	$y = x++$	y มีค่าเป็น 5

จากตัวอย่างที่ 4

ถ้า x มีค่าเป็น 7

$y = ++x$ แล้ว

อธิบายได้ว่า x มีค่าเป็น 7 ต่อมา เพิ่มค่า x ขึ้นหนึ่ง แล้วส่งให้ตัวแปร y ทำให้ y มีค่าเป็น 8

ถ้า x มีค่าเป็น 7

$y = x++$ แล้ว

อธิบายได้ว่า x มีค่าเป็น 7 ทำให้ y มีค่าเป็น 7 ด้วย และเพิ่มค่า x ขึ้นหนึ่ง ส่งผลให้ x มีค่าเป็น 8

สรุปได้ว่า

- ถ้าวางตัวดำเนินการไว้หน้าตัวแปร จะทำการเพิ่มค่าก่อน แล้วจึงส่งค่าให้กับ y

- ถ้าวางตัวดำเนินการไว้หลังตัวแปร จะทำการส่งค่าให้กับ y ก่อนแล้วจึงเพิ่มค่า x อีกหนึ่ง

การยุบนิพจน์โดยใช้ (Compound Assignment)

นิพจน์	ตัวดำเนินการที่ยุบแล้ว
$x = x + y$	$x += y$
$x = x - y$	$x -= y$
$x = x * y$	$x *= y$
$x = x / y$	$x /= y$
$x = x \% y$	$x \% = y$
$z = (x+1) + (y+1)$	$z = ++x + ++y$

ตัวดำเนินการเปรียบเทียบ (Relation Operator)

operator	ความหมาย	ชนิดของข้อมูล	ตัวอย่าง	ผลลัพธ์
<	น้อยกว่า	boolean	5 < 4	0
<=	น้อยกว่าหรือเท่ากับ	boolean	5 <= 5	1
==	เท่ากับ	boolean	0 == 0	1
>	มากกว่า	boolean	5 > 4	1
>=	มากกว่าหรือเท่ากับ	boolean	5 >= 4	1
!=	ไม่เท่ากับ	boolean	0 != 0	0

ตัวดำเนินการลอจิก (*Logic Operator*)

operator	ความหมาย	ชนิดของข้อมูล	ตัวอย่าง	ผลลัพธ์
&&	AND	boolean	1 && -1	1
	OR	boolean	1 0	1
!	NOT	boolean	!4	0

ตารางค่าความจริง **AND**

A	B	A && B
1	1	1
1	0	0
0	1	0
0	0	0

ตารางค่าความจริง **OR**

A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

ตารางค่าความจริง **NOT**

A	!A
1	0
0	1

ข้อมูลตรรกะ (*Boolean Data Type*)

■ ข้อมูลตรรกะ (Boolean Data Type)

จะเป็นค่าทางลอจิก

ชนิด	ตัวอย่าง
boolean	1 , 0 , -4

- ค่าเท็จ (False) แทนค่าด้วยเลข 0
- ค่าจริง (True) แทนค่าด้วยเลข 1
 - ค่าจริง คือ ค่าที่ไม่เท่ากับ 0



ตัวดำเนินการเปรียบเทียบ (Relation Operator)

โปรแกรมตัวอย่างที่ 21- ไฟล์ Main.java

//Example 21

```
public class Main
{
    public static void main(String[] args)
    {
        int x = 5;
        int y = 3;
        System.out.println(x == y); // returns false because 5 is not equal to 3
        System.out.println(x != y); // returns true because 5 is not equal to 3
        System.out.println(x > y); // returns true because 5 is greater than 3
        System.out.println(x < y); // returns false because 5 is not less than 3
        System.out.println(x >= y); // returns true because 5 is greater, or equal, to 3
        System.out.println(x <= y); // returns false because 5 is neither less than or equal to 3
    }
}
```

ผลการรันโปรแกรม

```
false
true
true
false
true
false
```



ตัวดำเนินการลอจิก (Logic Operator)

โปรแกรมตัวอย่างที่ 22- ไฟล์ Main.java

//Example 22

```
public class Main
{
    public static void main(String[] args)
    {
        int x = 5;
        System.out.println(x > 3 && x < 10); // returns true because 5 is greater than 3 AND 5 is less than 10
        System.out.println(x > 3 || x < 4); // returns true because one of the conditions are true (5 is greater
                                                // than 3, but 5 is not less than 4)
        System.out.println(!(x > 3 && x < 10)); // returns false because ! (not) is used to reverse the result
    }
}
```

ผลการรันโปรแกรม

true
true
false



Java Conditions

Syntax **if**

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Syntax **If-else**

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

Syntax **If-else-if**

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```



Java Conditions(ต่อ)

switch

Syntax

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```



Example Input โปรแกรมตัวอย่างที่ 23- ไฟล์ Main.java

```
1 //Example 23
2 import java.util.Scanner;
3
4 class Main
5 {
6     public static void main(String[] args)
7     {
8         Scanner myObj = new Scanner(System.in);
9
10        // String input
11        System.out.print("Enter name : ");
12        String name = myObj.nextLine();
13
14        // Numerical input
15        System.out.print("Enter age : ");
16        int age = myObj.nextInt();
17
18        System.out.print("Enter salary: ");
19        double salary = myObj.nextDouble();
20
21        // Output input by user
22        System.out.println("");
23        System.out.println("Name: " + name);
24        System.out.println("Age: " + age);
25        System.out.println("Salary: " + salary);
26    }
27 }
```

ผลการรันโปรแกรม

```
Enter name : peter
Enter age : 18
Enter salary: 1500

Name: peter
Age: 18
Salary: 1500.0
```



โจทย์

ให้นักศึกษาพัฒนาแอปพลิเคชันแสดงอักษรระดับคะแนน โดยใช้คำสั่งเงื่อนไข ให้สามารถรองรับการป้อนค่าจากคีย์บอร์ดโดยใช้คำสั่งรับ input รับค่าจำนวนเต็มจากผู้ใช้ ข้อกำหนดการทำงานของโปรแกรมมีดังนี้

โดยกำหนดให้ช่วงคะแนนมีดังนี้

คะแนน น้อยกว่า 0	ให้พิมพ์ “ERROR”	ออกมา
คะแนน มากกว่าเท่ากับ 0 แต่น้อยกว่า 50	ให้พิมพ์ “Grade F”	ออกมา
คะแนน มากกว่าเท่ากับ 50 แต่น้อยกว่า 55	ให้พิมพ์ “Grade D”	ออกมา
คะแนน มากกว่าเท่ากับ 55 แต่น้อยกว่า 60	ให้พิมพ์ “Grade D+”	ออกมา
คะแนน มากกว่าเท่ากับ 60 แต่น้อยกว่า 70	ให้พิมพ์ “Grade C”	ออกมา
คะแนน มากกว่าเท่ากับ 70 แต่น้อยกว่า 75	ให้พิมพ์ “Grade C+”	ออกมา
คะแนน มากกว่าเท่ากับ 75 แต่น้อยกว่า 80	ให้พิมพ์ “Grade B”	ออกมา
คะแนน มากกว่าเท่ากับ 80 แต่น้อยกว่า 85	ให้พิมพ์ “Grade B+”	ออกมา
คะแนน มากกว่าเท่ากับ 85 แต่น้อยกว่าเท่ากับ 100	ให้พิมพ์ “Grade A”	ออกมา
คะแนน มากกว่า 100	ให้พิมพ์ “ERROR”	ออกมา

เมื่อรันโปรแกรมจะมีผลการรันดังนี้

ถ้าป้อนค่ามากกว่าเท่ากับ 50 หรือ $\text{score} \geq 50$ จะมีผลการรันดังนี้

Please Enter Your Score : 70

Grade C



Java Loops

Syntax **for-loop**

```
for (statement 1; statement 2; statement 3) {  
    // code block to be executed  
}
```

Syntax **while-loop**

```
while (condition) {  
    // code block to be executed  
}
```

Syntax **do-while-loop**

```
do {  
    // code block to be executed  
}  
while (condition);
```



Q & A

